

Deploy do beca-list — Vercel + Render + Aiven (grátis)

Guia prático para publicar o projeto em produção, mantendo o funcionamento local (XAMPP) intacto.

Camada	Plataforma	O que hospeda
Frontend	Vercel	HTML/CSS/JS estático
Backend	Render	API Node.js + Express
Banco de dados	Aiven	MySQL (plano sempre gratuito)

```
[ Vercel (frontend) ] --fetch()--> [ Render  
(backend) ] --Sequelize (SSL)--> [ Aiven  
(MySQL) ]
```

1. Ajustes no projeto (para funcionar local e em produção)

Local (XAMPP) e produção (Render/Aiven) têm endereços e configurações diferentes. Em vez de trocar código na mão toda hora, o projeto deve **detectar sozinho** em qual ambiente está rodando.

1.1 Frontend — URL da API dinâmica

Hoje as chamadas `fetch()` provavelmente apontam direto para `http://localhost:3000`. Centralize isso em um único arquivo e detecte o ambiente pelo domínio:

```
// frontend/js/config.js
const API_BASE_URL = window.location.hostname
=== "localhost" || window.location.hostname ===
"127.0.0.1"
  ? "http://localhost:3000"
  : "https://SEU-BACKEND.onrender.com"; //
preencha depois do deploy do backend
```

E use `API_BASE_URL` em todas as chamadas do `api.js` (`fetch(`${API_BASE_URL}/tasks`)`). Assim, rodando local o front continua chamando o XAMPP/backend local, e publicado no Vercel ele chama o Render automaticamente — sem precisar comentar/descomentar nada.

1.2 Backend — porta dinâmica

O Render define a porta por conta própria através de uma variável de ambiente. O código não pode fixar a porta:

```
// server.js
const PORT = process.env.PORT || 3000; // 3000
é usado só quando rodar local
app.listen(PORT, () => console.log(`Servidor
rodando na porta ${PORT}`));
```

1.3 Backend — CORS liberando os dois domínios

```
// app.js
const cors = require("cors");

const allowedOrigins = [
  "http://127.0.0.1:5500", // frontend
  local (ajuste pra porta que você usar)
  "https://SEU-FRONTEND.vercel.app", //
  preencha depois do deploy do frontend
];

app.use(cors({ origin: allowedOrigins }));
```

1.4 Backend — SSL na conexão com o banco (só em produção)

O XAMPP local não exige SSL, mas o Aiven **exige**. Controle isso com uma variável de ambiente (`DB_SSL`), sem duplicar arquivos de configuração:

```
// src/config/database.js
require("dotenv").config();
const { Sequelize } = require("sequelize");

const useSSL = process.env.DB_SSL === "true";

const sequelize = new Sequelize(
  process.env.DB_NAME,
  process.env.DB_USER,
  process.env.DB_PASSWORD,
  {
    host: process.env.DB_HOST,
    port: process.env.DB_PORT || 3306,
    dialect: "mysql",
    dialectOptions: useSSL
```

```

    ? { ssl: { require: true,
rejectUnauthorized: false } } // simplificado,
suficiente para estudo
    : {},
  }
);

module.exports = sequelize;

```

Para uma configuração mais correta de SSL (validando o certificado de verdade), dá pra baixar o certificado CA do Aiven, colocar no repositório do backend e referenciar com `ca: fs.readFileSync(...)` no lugar de `rejectUnauthorized: false`. Para fins de estudo, a versão simplificada acima já funciona.

1.5 .env local x variáveis de ambiente em produção

O arquivo `.env` continua só para uso **local** (e continua fora do Git, no `.gitignore`). Em produção, as mesmas variáveis são cadastradas direto no painel do Render — não existe arquivo `.env` lá.

```

# .env (local, com XAMPP)
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=
DB_NAME=beca_list
DB_SSL=false

```

```
# Variáveis de ambiente no Render (produção, com Aiven)
DB_HOST=<host fornecido pelo Aiven>
DB_PORT=<porta fornecida pelo Aiven>
DB_USER=<usuário fornecido pelo Aiven>
DB_PASSWORD=<senha fornecida pelo Aiven>
DB_NAME=<nome do banco>
DB_SSL=true
```

2. Banco de dados — Aiven (MySQL grátis)

1. Crie uma conta em aiven.io (não pede cartão de crédito).
2. No console, clique em **Create service** → escolha **MySQL** → selecione o plano **Free**.
3. Escolha uma nuvem/região qualquer e dê um nome ao serviço (ex: `beca-list-db`).
4. Espere o serviço ficar com status **Running**.
5. Na página do serviço, em **Overview** → **Connection information**, copie: `Host`, `Port`, `User`, `Password` e o nome do banco padrão (ou crie um banco próprio em **Databases** → **Create database**).
6. Guarde esses dados — eles vão para as variáveis de ambiente do Render no próximo passo.

3. Backend — Render

1. Suba o projeto (pasta `beca-list/`, com `frontend/` e `backend/`) para um repositório no GitHub.
2. Em render.com, clique em **New +** → **Web Service** → conecte o repositório.
3. Configure:
 - **Root Directory:** `backend` (importante, já que o `backend` está numa subpasta do repositório)
 - **Environment:** `Node`
 - **Build Command:** `npm install`
 - **Start Command:** `node server.js` (ou `npm start`, se o `package.json` tiver esse script)
 - **Instance Type:** `Free`
4. Em **Environment Variables**, cadastre `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME` e `DB_SSL=true` com os dados copiados do Aiven.
5. Clique em **Create Web Service** e aguarde o build.
6. Ao final, copie a URL gerada (algo como `https://beca-list-backend.onrender.com`) e use ela no `config.js` do frontend (passo 1.1).

No plano free, o serviço "dorme" depois de um tempo sem uso e demora uns 30-60 segundos para acordar na primeira requisição — normal para um projeto de estudo.

4. Frontend — Vercel

1. Em vercel.com, clique em **Add New** → **Project** e importe o mesmo repositório do GitHub.
2. Configure:
 - **Root Directory:** `frontend` (a pasta com o `index.html`)
 - **Framework Preset:** `Other` (projeto sem framework/build)
3. Clique em **Deploy**.
4. Ao final, copie a URL gerada (algo como `https://beca-list.vercel.app`) e:
 - Atualize `allowedOrigins` no backend (passo 1.3) com essa URL.
 - Redeploy o backend no Render pra aplicar o CORS novo.

Alternativa ainda mais simples pra testar rápido, sem Git: arraste a pasta `frontend/` em vercel.com/drop — publica na hora, sem precisar de conta conectada ao GitHub. Pra deploys automáticos a cada `git push`, use o fluxo via GitHub (passos acima).

5. Checklist de deploy

- ☐ `config.js` do frontend detecta local x produção pela URL (seção 1.1)
- ☐ Backend usa `process.env.PORT` em vez de porta fixa (seção 1.2)

- [] CORS libera tanto o endereço local quanto o domínio do Vercel (seção 1.3)
 - [] Conexão com o banco usa SSL só quando `DB_SSL=true` (seção 1.4)
 - [] `.env` local continua fora do Git; credenciais de produção estão só nas variáveis de ambiente do Render
 - [] Banco criado no Aiven, com as tabelas geradas pelo próprio Sequelize (`sequelize.sync()`) na primeira execução em produção
 - [] Backend publicado no Render, rota `/tasks` responde direto pela URL pública
 - [] Frontend publicado no Vercel, apontando para a URL certa do backend
 - [] Projeto continua funcionando normalmente rodando local com XAMPP
-

6. Documentação oficial

Ferramenta	Link
Render — Deploy Node/Express	https://render.com/docs/deploy-node-express-app
Render — Variáveis de ambiente	https://render.com/docs/environment-variables

Ferramenta	Link
Vercel — Deploy de projetos estáticos	https://vercel.com/docs/deployments/overview
Aiven — MySQL sempre gratuito	https://aiven.io/free-mysql-database
Aiven — Conexão e SSL	https://aiven.io/docs/products/mysql
Sequelize — Opções de conexão (SSL)	https://sequelize.org/docs/v6/other-topics/dialect-specific-things/

Documento complementar à documentação principal do projeto beca-list, focado no processo de deploy em produção.